SAMORAH

User Scenario Document

USD v1.0 | 62 Complete Workflows

The single source of truth for how every user, admin, and system interacts with Samorah

Document Type	User Scenario Document (USD)
Version	1.0 — June 2025
BRD Reference	SAMORAH BRD v3.0
Total Flows	62 scenarios across 5 categories
Primary Use	Claude prompting — reference flow ID before each build task

HOW TO USE THIS DOCUMENT WITH CLAUDE

Before building any feature, tell Claude: 'Refer to SAMORAH USD flow [C3] before building the COD checkout flow.'

Each flow defines exactly what code is needed, what DB calls are made, what emails fire, and what edge cases must be handled.

Flows are color-coded: Green=Customer | Blue=Payment | Red=Admin | Purple=Marketing | Amber=Security

Flow Index — All 62 Scenarios

A. Customer Flows (C1–C22)

ID	Flow Name	Key Feature
C1	Guest First Visit & Purchase	Homepage → Product → Cart → Checkout → Razorpay → Confirmation
C2	Guest - Payment Failure + Retry	Razorpay failure → stock release → retry
C3	Guest - COD Order	Checkout → COD → Shiprocket → Delivered → Paid
C4	Google OAuth Login + Purchase	One-click Google → Cart merge → Purchase
C5	OTP Phone Login	Phone number → SMS OTP → Verified → JWT issued
C6	Forgot Password + Reset	Email → Secure link → New password → Login
C7	Email Verification (New Account)	Register → 6-digit OTP → Email verified → Access
C8	Wishlist Add + Later Purchase	Heart click → DB save → Return visit → Buy from wishlist
C9	Valid Coupon Application	Enter WELCOME20 → Validate → Discount applied → Checkout
C10	Coupon + Loyalty Stacking — REJECTED	Attempt both → System blocks → Clear error message
C11	Loyalty Points Earn (Post Purchase)	Order delivered → Points credited → Email → Balance update
C12	Loyalty Points Redeem at Checkout	Select points → Discount applied → Points deducted
C13	Tier Upgrade (Bronze → Silver)	500 points crossed → Tier badge updated → Upgrade email
C14	Gift Card Purchase + Delivery	Buy ₹1000 card → Razorpay → Code emailed to recipient
C15	Gift Card Partial Redemption	Apply code → ₹600 balance → ₹400 used → ₹200 remains
C16	Bundle Builder — 3 Candle Composition	Vessel filter → Pick 3 → Tray fills → Savings → Cart
C17	Referral Link Share + Referrer Reward	Copy link → Share → Friend buys → ₹100 coupon earned
C18	New User Uses Referral Code	ref= URL → Code captured → ₹100 off applied at first checkout

C19	Abandoned Cart — Full 3-Email Sequence	pg_cron → 1hr email → 24hr email → 72hr email with coupon
C20	Back-in-Stock Notification	Out of stock → 'Notify Me' → Restock → Auto email → Purchase
C21	Backorder Purchase	allow_backorder=true → 'Ships 7–10 days' shown → Order placed
C22	Gift Order with Gift Note + Occasion	Gift toggle → Note + recipient + Birthday → Packing slip

B. Payment & Order Flows (P1–P12)

ID	Flow Name	Key Feature
P1	Successful UPI Payment (Full Server Flow)	Frontend verify + Webhook as source of truth
P2	Card Payment with No-Cost EMI	EMI selection → Razorpay → Order placed → EMI tracked
P3	Net Banking Payment	Bank redirect → Return → Verify → Order confirmed
P4	COD Full Lifecycle	Place → Pack → Ship → Delivered → COD collected → Reconcile
P5	Client Drop-Off — Browser Closed Mid-Pay	Webhook rescues order without frontend verification
P6	Duplicate Webhook — Idempotency Protection	Second event → idempotency_key check → Blocked silently
P7	15-Minute Stock Reservation Expiry	Checkout start → Hold → No payment → pg_cron releases stock
P8	Order Tracking (Shipped → Delivered)	AWB → Shiprocket updates → DB updated → Customer emails
P9	Order Cancellation Pre-Shipment	Cancel request → Shiprocket cancel → Restock → Razorpay refund
P10	Return Request Post-Delivery	Complaint → Admin initiates → Refund → Optional restock
P11	GST Invoice — Intra-State (CGST + SGST)	Customer in Samorah's state → CGST 6% + SGST 6% = 12%
P12	GST Invoice — Inter-State (IGST)	Customer in different state → IGST 12% → Tax invoice PDF

C. Admin & Operations Flows (A1–A18)

ID	Flow Name	Key Feature
A1	Add New Candle Product (6-Step Form)	Name → Variants → Images → Notes → SEO → Publish → Live <60s
A2	Bulk CSV Product Upload (10–30 Products)	Template → Fill → Upload → Validate → Preview → Confirm
A3	Product Price Change + Audit Log	Edit price → Save → Audit: before/after JSONB logged
A4	Archive Product (Soft Delete)	Archive → Hidden from store → Reviews/orders preserved
A5	Process Incoming Order (Pack → Ship)	New order → Print packing slip → Pack → Status: Packed
A6	Create Shiprocket Shipment + Label	Packed → Create shipment → Assign courier → AWB → Print label
A7	NDR — Instruct Re-attempt Delivery	delivery_failed → Admin reviews → Calls customer → Re-attempt
A8	NDR — Accept RTO (Return to Origin)	3 failed attempts → Accept RTO → Restock → Refund
A9	Inventory Manual Restock	Low stock alert → Adjust stock → Reason note → Audit log
A10	Bulk CSV Inventory Update	Upload variant_sku + new_stock CSV → Validate → Preview → Apply
A11	Create and Launch Coupon	Code → Type → Value → Rules → Active → Share in campaign
A12	Generate Gift Card (Admin)	Amount → Recipient → Generate code → Send branded email
A13	Loyalty Points Manual Adjustment	Customer → Admin adjusts → Reason → Transaction logged
A14	Publish Blog Post with SEO + Schedule	Write → SEO fields → Schedule for 10am → Auto-published
A15	Update Homepage Banner (No Code)	Admin Banners → Edit hero slot → Save → ISR triggers → Live
A16	Instagram Gallery Update	Upload lifestyle images → Set featured 9 → Reorder → Save
A17	Wholesale Customer Approval	Form submitted → Admin reviews → Approve → Pricing tier set
A18	Admin Role Change + Audit	Super Admin → Find user → Change role → Audit log entry

D. Marketing & Automation Flows (M1–M12)

ID	Flow Name	Key Feature
M1	New Collection Launch Campaign	Teaser → Products live → Blog → Banner → Email → GA4 spike
M2	Diwali Festival Campaign	Segment list → Create campaign → DIWALI25 coupon → Schedule → Send
M3	Birthday Campaign (pg_cron Daily)	pg_cron finds birthdays → 100 bonus points credited → Email
M4	Newsletter Campaign Send	Segment → Template → Preview test → Schedule → Open rate monitor
M5	Abandoned Cart Email Sequence	pg_cron hourly → 1hr → 24hr → 72hr with coupon → Purchase
M6	Post-Purchase Review Request	Delivered → Day 3: review email → Day 30: replenishment nudge
M7	Welcome Email Sequence	Register → Immediate → Day 3 brand story → Day 7 chapters
M8	Low Stock Alert Email (Admin)	Stock hits threshold → pg_cron → Resend admin alert email
M9	Back-in-Stock Campaign	Restock → notify_me subscribers email → Track clicks → Conversions
M10	Referral Reward Email Trigger	Referred user first order → Referrer gets coupon email auto
M11	Tier Upgrade Notification	Points cross threshold → Tier updated → Upgrade email + perks
M12	Search Analytics — Zero Result Query	Customer types unknown term → 0 results → search_logs entry

E. Security & Edge Case Flows (S1–S8)

ID	Flow Name	Key Feature
S1	Coupon + Loyalty Point Stacking — Blocked	Both applied → API rejects → Clear UI error → One allowed
S2	Browser Close Mid-Payment — Webhook Rescue	Frontend never calls /verify → Webhook fires → Order saved
S3	Duplicate Razorpay Webhook — Silent Block	Same payment_id received twice → idempotency check → 200 OK
S4	Stock Reservation Timeout (15 min)	Checkout started → No payment → pg_cron releases → Back in stock
S5	Unauthorized Admin Access Attempt	Customer hits /admin → Middleware JWT check → 403 redirect

S6	Rate Limiting — Brute Force Login	5 failed logins → Rate limiter → 429 response → 15min lockout
S7	Webhook Log — Razorpay Error Debug	Failed webhook → webhook_logs entry → Admin can inspect payload
S8	Product Recommendations — Fragrance Family	View Kashmiri Chai → related_products from same fragrance family

SECTION A — Customer Flows (C1–C22)

C1 Guest First Visit & Purchase

Customer Flow — Homepage to Order Confirmed

Actor	Guest User (no account)	**Trigger**	Arrives on samorah.in from Google/Instagram/Direct
Preconditions	No account, no saved cart, no saved addresses		

Step	Action (Who Does What)	System Response	DB / API Call
1	Browser opens samorah.in	ISR homepage loads <1.5s. Hero, announcement bar, chapter slider rendered. Framer Motion fade-in plays.	GET / (ISR cached)
2	Hovers over SHOP in mega menu	Mega menu slides open — candle categories + campaign image + chapters list.	—
3	Clicks 'Vol. I — Dessert Chapter'	Collection editorial page loads. Hero → Featured Kashmiri Chai → Supporting grid (Modak, Gajar Halwa).	GET /collections/dessert-chapter
4	Clicks 'Kashmiri Chai' product card	Product page loads (SSG). Gallery, fragrance pyramid, variant selector, Add to Cart visible.	GET /shop/kashmiri-chai
5	Selects '180g Ceramic' variant	Price updates to ₹649. In-stock badge shown. 'Add to Cart' button activates.	useCartStore — variant state
6	Clicks 'Add to Cart'	Mini cart opens (useUIStore). Item added to Zustand cartStore + persisted to localStorage. Toast: 'Added to cart'.	cartStore.addItem() + localStorage
7	Clicks mini cart → 'View Cart'	Cart page: 1 item, ₹649 subtotal. Coupon field shown. Gift note option shown. Shipping estimate shown.	GET /cart (CSR)
8	Clicks 'Proceed to Checkout'	Guest checkout: email + delivery address form. No login required.	GET /checkout

9	Enters email + Delhi address (pincode 110001)	Shiprocket pincode API checks serviceability. Returns: Standard ₹60, Express ₹120, COD: Available.	POST /api/shiprocket/pincode-check
10	Selects Standard Delivery	Delivery cost added. Order total: ₹649 + ₹60 = ₹709. GST shown (IGST 12% = ₹69.75 included in price).	checkoutStore.setDelivery()
11	Clicks 'Proceed to Payment'	Review screen: items, address, total breakdown, GST breakdown. 'Place Order' CTA.	—
12	Clicks 'Place Order'	POST /api/razorpay/create-order → Razorpay order created (₹70900 paise). orders row: status=pending, payment_status=pending.	POST Razorpay API + INSERT orders
13	Razorpay overlay opens — selects GPay UPI	Customer authorizes payment in GPay app.	Razorpay SDK
14	Payment deducted from bank	Razorpay sends payment.captured webhook to /api/razorpay/webhook (server-to-server).	POST /api/razorpay/webhook
15	Webhook: idempotency check	API: SELECT * FROM orders WHERE idempotency_key = payment_id → No match → Process.	SELECT orders WHERE idempotency_key
16	Webhook: update order	orders.status = confirmed, payment_status = paid, idempotency_key = payment_id. Inventory: variants.stock -= 1.	UPDATE orders + UPDATE variants
17	Webhook: Shiprocket order created	POST Shiprocket /orders/create. shiprocket_order_id stored.	POST Shiprocket API
18	Webhook: Tax Invoice PDF generated	IGST calculated. Invoice PDF: TAX INVOICE, SAM/2025-26/0001, HSN 3406, IGST 12%.	generateInvoicePDF()
19	Webhook: Resend email sent	'Order Confirmed' email: items, ₹709 total, Tax Invoice PDF attached, estimated delivery.	POST Resend API
20	Frontend: /verify called (redirect)	POST /api/razorpay/verify → HMAC verify succeeds →	HMAC verify + GET order

		Redirects to /checkout/success/[orderId].	
21	Customer sees order confirmation page	Order number SAM-2025-0001. Estimated delivery 5–7 days. Invoice download link. 'Continue Shopping' CTA.	GET /checkout/success/[id]

Error / Edge Case	Handling
Customer closes browser after paying but before /verify	Webhook fires regardless. Order is saved, inventory decremented, email sent. Customer can find order at /account with email lookup.
UPI payment times out	Razorpay sends payment.failed webhook → order status = failed → stock reservation released → failure email → retry CTA on page.
Shiprocket pincode not serviceable	Checkout: 'Delivery not available to this pincode' message. Customer can update address.
Duplicate webhook received	idempotency_key already exists → 200 OK returned immediately → no duplicate processing.

Claude Build Note:
Build in this sequence: (1) cartStore with localStorage + hydration-safe useStore hook, (2) /api/razorpay/create-order route, (3) /api/razorpay/webhook with idempotency check FIRST before inventory/email logic, (4) confirmation page that reads order from DB (not from frontend state).

C2 Guest — Payment Failure + Retry

Customer Flow — Handling Razorpay Failures

Actor	Guest User	**Trigger**	Customer's UPI payment fails or times out
Preconditions	Cart with items, checkout address filled, Razorpay order created		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer is on Razorpay overlay	Razorpay order created, stock reservation active (15-min hold started).	orders.status = pending
2	UPI app shows 'Payment Failed' or times out	Razorpay sends payment.failed webhook to /api/razorpay/webhook.	POST /api/razorpay/webhook (failed event)
3	Webhook processes failure	orders.status = failed, payment_status = failed. Stock reservation released (variants.stock back to original).	UPDATE orders + release reservation
4	Failure email sent	Resend: 'Payment unsuccessful' email with order total and 'Try Again' CTA link.	POST Resend API
5	Frontend: Razorpay overlay shows error	Customer sees 'Payment Failed' in overlay. Razorpay overlay has built-in retry.	Razorpay SDK
6	Customer clicks 'Retry'	New Razorpay order created (new razorpay_order_id). Previous failed order remains with status=failed.	POST /api/razorpay/create-order (new)
7	Customer selects different method (e.g. Card)	Completes payment successfully.	payment.captured webhook fires
8	New order confirmed	New order row (SAM-2025-0002 if original failed). Inventory decremented. Email sent.	Standard C1 confirmation flow

Error / Edge Case	Handling
Customer retries 3 times and all fail	Each attempt creates a new Razorpay order. All failed orders remain in DB with status=failed for records.

Bank debits but Razorpay shows failure	This is a Razorpay settlement issue. payment.captured webhook will fire when confirmed. Idempotency check protects.
Customer leaves without retrying	Cart remains in localStorage (guest). Next visit cart is restored automatically.

Claude Build Note:

Ensure payment.failed webhook releases the stock_reservations row or sets quantity=0. Never leave stock held indefinitely after failure. The pg_cron job (every 15 min) is the safety net for any reservations where webhook was missed.

C3 Guest — COD Order (Cash on Delivery)

Customer Flow — COD Full Lifecycle

Actor	Guest User	**Trigger**	Customer selects Cash on Delivery at checkout
Preconditions	Pincode is COD-serviceable per Shiprocket API		

Step	Action (Who Does What)	System Response	DB / API Call
1	Pincode check at checkout	POST /api/shiprocket/pincode-check returns COD: Available.	Shiprocket pincode API
2	Customer selects COD at payment step	No Razorpay overlay. Order summary shown. 'Place Order (COD)' button.	—
3	Customer clicks 'Place Order (COD)'	orders row: status=confirmed, payment_status=pending (COD). No Razorpay order_id needed.	INSERT orders (COD type)
4	Order confirmation email sent	'Order Confirmed (COD)' email: items, total, delivery estimate, 'Pay ₹709 to courier on delivery'.	POST Resend API
5	Admin sees new COD order in dashboard	Orders list shows COD badge. No Razorpay payment ID.	Admin order list
6	Admin packs order	Status updated to packed. Packing slip printed — shows COD amount clearly.	UPDATE orders.status = packed
7	Admin creates Shiprocket COD shipment	POST Shiprocket with payment_method: COD. AWB generated. Shipping label printed.	POST Shiprocket API (COD)
8	Order shipped	Status → shipped. 'Order Shipped' email sent with AWB tracking link.	UPDATE + Resend
9	Courier delivers — customer pays cash	Shiprocket tracking → delivered. orders.payment_status = paid (COD collected).	Shiprocket webhook → UPDATE
10	COD reconciliation	Shiprocket shows COD remittance in their dashboard.	Admin → Orders → COD

		Admin marks payment received.	

Error / Edge Case	Handling
Customer refuses delivery (common in India)	Shiprocket sends NDR webhook. ndr_status = delivery_failed. Admin sees in NDR tab. Must call customer.
Customer unavailable — delivery failed	Same NDR flow. Admin can instruct re-attempt or accept RTO.
COD order cancelled before pickup	Admin cancels Shiprocket order. Stock restored. No refund needed (no payment made).

Claude Build Note:
COD orders should skip Razorpay entirely. The checkout API route needs a branch: if payment_method === 'cod' → skip Razorpay order creation → insert order directly with payment_status = pending_cod. Make sure packing slips for COD clearly show the 'Collect ₹[amount] from customer' instruction.

C9 Valid Coupon Application

Customer Flow — Coupon Code Validation & Discount

Actor	Registered or Guest Customer	**Trigger**	Customer enters coupon code WELCOME20 in cart
Preconditions	Cart has items totaling ₹649. WELCOME20 exists: 20% off, min order ₹500, max 100 uses, expires Dec 31, is_active=true		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer enters 'WELCOME20' in coupon field	Real-time input — no API call yet. 'Apply' button shown.	—
2	Customer clicks 'Apply'	POST /api/coupons/apply: { code: 'WELCOME20', cart_total: 649, user_id: null }	POST /api/coupons/apply
3	API validates coupon	1) Exists? Yes. 2) Active? Yes. 3) Expired? No. 4) Min order met? Yes (649>500). 5) Usage limit? 45/100 used. 6) User already used? No. 7) First-order-only? No. All pass.	SELECT coupons WHERE code
4	Discount calculated	20% of ₹649 = ₹129.80. New total: ₹649 − ₹129.80 + ₹60 shipping = ₹579.20	Client-side calc
5	cartStore updated	coupon: { code:'WELCOME20', discount: 129.80 }. UI updates: crossed-out original price, green discount line, new total.	cartStore.applyCoupon()
6	Customer proceeds to checkout	Coupon applied through checkout. Discount stored in checkoutStore.	—
7	Order placed and paid	On payment.captured webhook: coupons.used_count += 1. Discount stored in orders.discount_amount.	UPDATE coupons + INSERT order

Error / Edge Case	Handling
Coupon doesn't exist	API returns error: 'Invalid coupon code'. UI shows red error under field.
Coupon expired	API returns: 'This coupon has expired.' UI error shown.

Minimum order not met	API returns: 'Minimum order of ₹500 required.' (with the amount shown)
Usage limit reached	API returns: 'This coupon is no longer available.'
Customer already used this coupon	If single-use or user-specific coupon, API returns: 'You have already used this coupon.'

Claude Build Note:
The coupon validation API must perform ALL 7 checks atomically. Never decrement used_count at apply time — only on successful order completion via webhook. This prevents count inflation if customers abandon after applying.

C10 Coupon + Loyalty Points — Stacking BLOCKED

Customer Flow — Discount Stacking Rules Enforced

⚠ DISCOUNT STACKING RULE — Must be enforced at API level:
RULE 1: Only ONE coupon code can be applied per order. A second code replaces the first.
RULE 2: Coupons CANNOT be combined with Loyalty Point redemptions.
RULE 3: Gift Card codes are EXEMPT from this rule — they can be combined with either coupons OR loyalty points.
Rationale: Prevents zero/negative order totals. Gift cards are pre-paid currency, not discounts.

Actor	Registered Customer	**Trigger**	Customer tries to apply WELCOME20 coupon AFTER already having 500 loyalty points queued for redemption
Preconditions	Cart: ₹649. loyaltyPoints queued: 500pts (=₹50 off). Coupon WELCOME20: 20% off.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer is on cart page	Cart shows: Subtotal ₹649. 'Redeem Points' section shows 500 pts available.	—
2	Customer clicks 'Redeem 500 Points'	API: validates points balance. cartStore: loyalty_discount = 50. UI: '₹50 off (500 points)' shown.	GET /api/loyalty/validate
3	Customer enters WELCOME20 in coupon field and clicks Apply	POST /api/coupons/apply is called.	POST /api/coupons/apply
4	API detects loyalty redemption active	API checks: cartStore has loyalty_discount > 0 (or checkoutStore.loyaltyPointsUsed > 0). Combination rule violated.	Server-side check
5	API returns stacking error	HTTP 409: { error: 'discount_stack_conflict', message: 'A coupon code cannot be combined with loyalty point redemptions. Please choose one.' }	Error response

6	UI shows clear choice modal	Modal: 'Choose one discount: [Keep ₹50 loyalty discount] or [Apply WELCOME20 (save ₹129.80)]'. Two buttons.	useUIStore.openModal()
7	Customer chooses WELCOME20	Loyalty points redemption cleared (cartStore.loyalty_discount = 0, loyaltyPointsUsed = 0). Coupon applied. Discount = ₹129.80.	cartStore update
8	OR: Customer chooses loyalty points	Coupon not applied. Loyalty discount of ₹50 remains. Customer proceeds.	cartStore unchanged

Error / Edge Case	Handling
Customer tries to apply 2 coupon codes	Second code replaces first. API returns: 'Coupon WELCOME20 replaced FIRST10.' Previous coupon discount removed.
Customer redeems points on checkout but coupon was in cart	Checkout API must re-validate: if loyalty points redemption attempted, reject if coupon already applied.
Gift card + coupon (allowed)	Gift card is pre-paid currency. POST /api/gift-cards/apply can co-exist with a coupon. No stacking conflict.
Gift card + loyalty points (allowed)	Same rule — gift card does not trigger stacking check.

Claude Build Note:
Enforce stacking rule in BOTH the /api/coupons/apply endpoint AND the /api/checkout/finalize endpoint (server-side double-check). Never rely on frontend-only enforcement. The checkout finalize must reject the order if both are present. UI modal approach gives customers a good experience instead of a hard error.

C16 Bundle Builder — 3 Candle Composition

Customer Flow — Interactive Bundle Builder

Actor	Any Customer	**Trigger**	Customer opens /bundles to build a custom candle set
Preconditions	Bundle products exist in DB with bundle-eligible flag. Minimum 3 different candles available per vessel type.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer opens /bundles	Bundle builder loads. Left: 3-slot empty composition tray + vessel filter buttons (Glass | Ceramic | Terracotta). Right: full candle catalog.	GET /bundles (CSR)
2	Customer clicks 'Ceramic' vessel filter	Right panel filters: only ceramic-vessel variants shown. Each card: candle name, scent notes (Bergamot • Neroli), price, 'Add to Composition' button.	productService.getVariantsByVessel('ceramic')
3	Customer clicks 'Add to Composition' on Kashmiri Chai	Left tray: '1/3 Selected | 2 more candles required'. Kashmiri Chai appears in tray slot 1. Button on right → 'Added' (muted, not clickable again).	bundleStore.addItem()
4	Customer clicks 'Add to Composition' on Modak	Left tray: '2/3 Selected | Ready to Add to Cart'.	bundleStore.addItem()
5	Customer clicks 'Add to Composition' on Gajar Halwa Delight	Left tray: '3/3 Selected | Curated Bundle Ready (italic)'. ALL remaining 'Add to Composition'	bundleStore.addItem()

		buttons → 'Composition Complete' (muted). Savings shown.	
6	Savings displayed in tray	Regular Value: ₹1,947. Bundle Price: ₹1,497. Bundle Saving: ₹450. 'Add Bundle to Cart' button active.	Client calculation
7	Customer clicks 'Add Bundle to Cart'	Bundle added as SINGLE cart item with 3 sub-items. Bundle SKU: SAM-BND-[auto]. Discount stored. Mini cart opens.	cartStore.addBundle()
8	Customer proceeds to checkout	Bundle treated as one line item in checkout. Tax applies at 12% on bundle total. Shiprocket weight = sum of 3 candles.	Checkout flow same as C1

Error / Edge Case	Handling
One component candle goes out of stock mid-session	'Add to Composition' button for that candle shows 'Out of Stock'. If already in tray, warning shown: 'Kashmiri Chai is no longer available. Please replace.'
Customer tries to select same candle twice	Prevented — once added to tray, the button state changes to 'Added'. DB constraint on bundle line items prevents duplicate product_id.
Customer changes vessel filter after partial selection	Warning modal: 'Changing vessel filter will clear your current composition. Continue?' Two buttons: Yes (clear) / No (keep).

Claude Build Note:
Bundle builder needs its own Zustand bundleStore (separate from cartStore) to manage the composition state. When 'Add Bundle to Cart' is clicked, create one cart item with a nested items array in the variant data. The checkout API must expand this for Shiprocket (uses combined weight) and inventory (decrements each component SKU individually).

C19 Abandoned Cart — Full 3-Email Sequence

Customer Flow — pg_cron Automated Recovery

Actor	Guest or Registered Customer	**Trigger**	Customer adds to cart but does not complete checkout
Preconditions	Cart table has row with last_activity_at timestamp. No order placed in the session.		

Trigger Mechanism (pg_cron — runs every hour):

```
SELECT c.session_id, c.user_id, c.items, u.email FROM cart c LEFT JOIN users u ON c.user_id = u.id WHERE c.last_activity_at < NOW() - INTERVAL '1 hour' AND NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = c.user_id AND o.created_at > c.last_activity_at) AND c.abandoned_email_1_sent = false;
```

Step	Action (Who Does What)	System Response	DB / API Call
1	Hour 0: Customer adds Kashmiri Chai to cart	cart.last_activity_at = NOW(). Zustand cartStore + localStorage.	INSERT/UPDATE cart row
2	Customer closes browser without buying	No order created. cart row remains.	—
3	Hour 1: pg_cron runs — detects abandonment	Query finds cart. abandoned_email_1_sent = false → triggers Email 1.	pg_cron query
4	Email 1 sent: 'Your candles are waiting'	Subject: 'Your candles are waiting for you ✦'. Content: product image, name, scent notes, price. Soft emotional tone. No discount. CTA: 'Return to Cart'. cart.abandoned_email_1_sent = true.	POST Resend API
5	Hour 24: pg_cron runs again	cart.abandoned_email_2_sent = false → triggers Email 2.	pg_cron query
6	Email 2 sent: 'Still thinking about us?'	Subject: 'Still thinking about us?'. Content: candle story excerpt, fragrance note description. No discount. CTA: 'Complete Your Order'.	POST Resend API

		cart.abandoned_email_2_sent = true.	
7	Hour 72: pg_cron runs again	cart.abandoned_email_3_sent = false → triggers Email 3.	pg_cron query
8	Email 3 sent: 'A small gift for you'	Subject: 'We saved something for you'. 10% discount coupon auto-generated (CART10-[uuid], single use, 48hr expiry). CTA: 'Claim 10% Off'. cart.abandoned_email_3_sent = true.	POST Resend + INSERT coupons
9	Customer clicks link in email	Cart restored from DB (for registered users) or localStorage (for guests). Coupon CART10-xxx pre-applied.	GET /cart
10	Customer completes purchase	Order placed. cart row deleted. All abandoned_email flags irrelevant.	DELETE cart row on order.confirmed

Error / Edge Case	Handling
Guest abandoned cart (no email captured)	No email to send to. pg_cron skips guest carts without email. MITIGATION: Show 'Save cart to email' prompt in cart page.
Customer has already purchased another product in between	pg_cron query checks: orders WHERE created_at > cart.last_activity_at. If found → skip. No email sent.
Customer unsubscribed from marketing	Check newsletter.is_active = false before sending abandoned cart emails. Still send transactional (order confirm) but not marketing sequences.
Duplicate emails (pg_cron runs twice)	Prevented by abandoned_email_1_sent, _2_sent, _3_sent boolean flags. Each email type sent exactly once.

Claude Build Note:

pg_cron must be enabled in Supabase: CREATE EXTENSION IF NOT EXISTS pg_cron; Then: SELECT cron.schedule('abandoned-cart-sweep', '0 * * * *', 'SELECT sweep_abandoned_carts()'); Create a PostgreSQL function sweep_abandoned_carts() that handles the query and Resend API call via a Supabase Edge Function trigger.

C22 Gift Order — Gift Note, Recipient & Occasion

Customer Flow — Gift Purchase Experience

Actor	Registered or Guest Customer	**Trigger**	Customer is buying a candle as a gift for someone else
Preconditions	Cart has items. Customer is on /cart page.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer sees 'This is a gift' toggle in cart	Toggle is visible below cart items. Default: OFF.	—
2	Customer toggles 'This is a gift' ON	Gift section expands: Gift Note (textarea), Recipient Name (optional), Occasion (dropdown).	cartStore.setIsGift(true)
3	Customer fills gift note: 'Happy Birthday, Priya!'	Max 200 chars. Character count shown. Stored in cartStore.giftNote.	cartStore.setGiftNote()
4	Customer selects Recipient: 'Priya Sharma'	Stored in cartStore.giftRecipient.	—
5	Customer selects Occasion: 'Birthday'	Dropdown: Birthday | Anniversary | Diwali | Christmas | Mother's Day | Just Because | Other. Stored in cartStore.giftOccasion.	—
6	Customer proceeds to checkout	Gift data passed to checkoutStore: { isGift: true, giftNote: '...', giftRecipient: 'Priya Sharma', giftOccasion: 'Birthday' }	—
7	Order placed and confirmed	orders.gift_note, orders.gift_recipient, orders.gift_occasion saved.	INSERT orders with gift fields
8	Admin prints packing slip	Packing slip shows: 'GIFT ORDER' banner at top. Pricing HIDDEN. Gift note prominently displayed: 'Happy Birthday, Priya!' Recipient name shown.	Packing slip PDF template
9	Order confirmation email	Customer's email: shows gift note. 'You sent a gift to Priya	POST Resend API

		Sharma (Birthday).' No gift note sent to recipient (they receive the physical surprise).	

Error / Edge Case	Handling
Customer forgets to toggle gift but adds a note	If giftNote field is filled, system auto-treats as gift even if toggle wasn't explicitly set.
Customer wants pricing removed from packing slip	When isGift = true, packing slip PDF template automatically hides all pricing.

Claude Build Note:
The packing slip PDF needs TWO templates: standard (shows pricing) and gift (hides pricing, shows gift note prominently). The admin order detail page should show a 'GIFT ORDER' badge prominently so warehouse staff know to use the gift packing slip template.

SECTION B — Payment & Order Flows (P1–P12)

P5 Client Drop-Off — Browser Closed Mid-Payment

Payment Flow — Webhook as Source of Truth

CRITICAL ARCHITECTURE DECISION:

The frontend /api/razorpay/verify endpoint exists ONLY to redirect the user to the Thank You page.

ALL business logic (inventory decrement, Shiprocket, email, invoice) MUST run exclusively in the server-to-server webhook at /api/razorpay/webhook.

Never put inventory or order update logic in /verify. That endpoint may never be called if the browser closes.

Actor	Guest Customer	**Trigger**	Customer's phone dies / internet drops the exact moment UPI payment is deducted
Preconditions	Payment was deducted from customer's bank. Razorpay has a payment_id. Frontend never calls /verify.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer reaches Razorpay overlay	Razorpay order_id created. orders.status = pending.	POST /api/razorpay/create-order
2	Customer pays via UPI — bank debits ₹709	Success in bank. Razorpay captures the payment. payment_id generated.	Razorpay internal
3	Customer's phone dies	Frontend is gone. /api/razorpay/verify is NEVER called.	—
4	Razorpay sends payment.captured webhook to server	Server-to-server HTTP POST to /api/razorpay/webhook. This happens regardless of frontend.	POST /api/razorpay/webhook
5	Webhook: HMAC signature verified	sha256(razorpay_order_id + '|' + razorpay_payment_id, key_secret) must match. If mismatch → 401, discard.	HMAC verify
6	Webhook: idempotency check	SELECT * FROM orders WHERE idempotency_key =	SELECT orders

		payment_id → No match → proceed.	
7	Webhook: Update order	status = confirmed, payment_status = paid, idempotency_key = payment_id.	UPDATE orders
8	Webhook: Inventory decremented	variants.stock -= 1 for the ordered SKU.	UPDATE variants
9	Webhook: Shiprocket order created	POST Shiprocket. shiprocket_order_id stored.	POST Shiprocket API
10	Webhook: GST calculated + invoice generated	IGST/CGST/SGST stored in orders. Tax Invoice PDF generated.	generateInvoicePDF()
11	Webhook: Order confirmation email sent	Customer receives email with order details and invoice. Order is saved.	POST Resend API
12	Customer charges phone next day	Opens email → sees order confirmation SAM-2025-0001 → all good. They can track at /account.	—

Error / Edge Case	Handling
Webhook also fails (network error on Samorah's server)	Razorpay retries webhook up to 8 times over 24 hours. idempotency_key check ensures safe retry.
Customer contacts support 'I paid but no confirmation'	Admin searches by email in Orders → finds order with payment_status=paid → resends confirmation email manually.
Frontend /verify called after webhook already processed	Both execute: /verify checks HMAC and reads the updated order → redirects to success. No double-processing because webhook already set idempotency_key.

Claude Build Note:

Critical implementation rule: /api/razorpay/verify should ONLY do: (1) HMAC verify, (2) read order from DB to confirm status=paid, (3) return redirect URL. NOTHING ELSE. Put ALL side effects (inventory, Shiprocket, email, GST) in /api/razorpay/webhook. Test this by commenting out /verify entirely — the order should still be fully processed.

P6

Duplicate Webhook — Idempotency Protection

Payment Flow — Preventing Double Processing

Actor	System (Razorpay → Samorah server)	**Trigger**	Razorpay sends payment.captured webhook, then retries because it didn't receive a 200 response in time
Preconditions	Payment is successfully captured once. orders.idempotency_key stores the payment_id after first processing.		

Step	Action (Who Does What)	System Response	DB / API Call
1	First webhook received: payment.captured	payload: { payment_id: 'pay_ABC123', order_id: 'order_XYZ' }	POST /api/razorpay/webhook
2	HMAC signature verified	Valid signature confirmed.	HMAC verify
3	Idempotency check	SELECT * FROM orders WHERE idempotency_key = 'pay_ABC123' → 0 rows. This is first processing.	SELECT orders
4	Order processed	status=paid, inventory decremented, Shiprocket created, email sent, idempotency_key = 'pay_ABC123'.	Full processing + UPDATE
5	Response: 200 OK sent to Razorpay	Razorpay marks webhook as delivered successfully.	HTTP 200
6	2 hours later: Razorpay retries (network issue earlier)	Same payload: { payment_id: 'pay_ABC123' }.	POST /api/razorpay/webhook
7	HMAC signature verified	Valid again.	HMAC verify
8	Idempotency check	SELECT * FROM orders WHERE idempotency_key = 'pay_ABC123' → 1 row found!	SELECT orders
9	Early return — no processing	Order already processed. Return 200 OK immediately. No inventory change. No email. No Shiprocket.	HTTP 200 (no side effects)

Error / Edge Case	Handling
Idempotency key not set before downstream actions	If server crashes between setting key and completing actions, retry will reprocess. FIX: Set idempotency_key as the FIRST write, use DB transaction for all writes.
Shiprocket webhook also needs idempotency	Create shiprocket_webhook_logs table. Same pattern: check if tracking_id already processed.

Claude Build Note:

Use a PostgreSQL transaction for the entire webhook processing: BEGIN; UPDATE orders SET idempotency_key=..., status='paid' WHERE id=... AND idempotency_key IS NULL; IF NOT FOUND → already processed, ROLLBACK, return 200; ELSE continue with inventory/Shiprocket/email; COMMIT;

P7 Stock Reservation — 15-Minute Hold & Expiry

Payment Flow — Preventing Overselling

Actor	Customer + pg_cron System	**Trigger**	Customer begins checkout — stock must be held to prevent overselling
Preconditions	Variant has stock=3. Customer selects 180g Ceramic. Another customer may also be checking out.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer clicks 'Proceed to Checkout'	POST /api/checkout/reserve: { variant_id, quantity: 1, session_id }	POST /api/checkout/reserve
2	API creates stock reservation	INSERT stock_reservations: { variant_id, quantity: 1, session_id, expires_at: NOW() + 15 min }	INSERT stock_reservations
3	Available stock check in product page	SELECT variants.stock - SUM(stock_reservations.quantity) WHERE expires_at > NOW(). Returns: 2 available (3 minus 1 held).	SELECT with reservation deduction
4	Customer fills address, selects delivery, opens Razorpay	Time passes. 14 minutes used.	—
5	Payment completes at 16 minutes (reservation expired)	pg_cron runs every 5 min: DELETE FROM stock_reservations WHERE expires_at < NOW(). Reservation deleted.	pg_cron DELETE expired
6	Webhook tries to decrement inventory	UPDATE variants SET stock = stock - 1 WHERE id = variant_id AND stock >= 1. Returns 1 row → success! Stock was 3, reservation expired but stock was still 3.	UPDATE variants
7	Order confirmed — no oversell	Stock correctly decremented. No issue even though reservation expired.	—

8	Scenario: Stock=1, Two customers checkout simultaneously	Both reserve stock_reservation. Second customer's reservation fails (qty check): if stock - active_reservations < 1 → reject checkout. Second customer sees 'Out of stock during checkout — sorry!' message.	Reservation conflict handling

Error / Edge Case	Handling
pg_cron doesn't run (DB job failure)	Stock reservations accumulate. FIX: Always calculate available = stock - active_non_expired_reservations in display logic. Fallback: on checkout initiation, clear own expired reservation.
Customer pays after reservation expires but before another customer	Payment webhook decrements stock with: UPDATE WHERE stock >= 1. If stock=0, update affects 0 rows → trigger oversell alert to admin.

Claude Build Note:

Create a stock_reservations table: id, variant_id, quantity, session_id (or user_id), expires_at, created_at. The 'available stock' shown on product pages = variants.stock - SUM(active reservations). Never show raw stock column to frontend. This single rule prevents all overselling scenarios.

P11 + P12 GST Calculation — Intra-State & Inter-State

Payment Flow — Indian Tax Law Compliance

GST Rules for Samorah:
Samorah's registered state: Maharashtra (example — update to actual state in settings.store_state)
Intra-state order (Customer in Maharashtra): CGST 6% + SGST 6% = 12% total
Inter-state order (Customer in any other state): IGST 12% total
GST is INCLUSIVE in the displayed MRP. Do not add GST on top of price — extract it from MRP.

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer in Delhi places order for ₹649 candle	Delhi ≠ Maharashtra (Samorah's state) → Inter-state → IGST applies.	settings.store_state vs delivery.state
2	GST extracted from MRP (GST-inclusive pricing)	Taxable value = ₹649 / 1.12 = ₹579.46. IGST = ₹649 - ₹579.46 = ₹69.54. Stored: orders.igst_amount = 69.54, cgst_amount = 0, sgst_amount = 0.	calculateGST() function
3	Tax Invoice PDF generated	Shows: Taxable Value ₹579.46 | IGST (12%) ₹69.54 | Total ₹649. HSN: 3406.	generateInvoicePDF()
4	Customer in Mumbai places same order	Mumbai = Maharashtra = Samorah's state → Intra-state → CGST + SGST.	settings.store_state match
5	GST extracted for intra-state	Taxable value = ₹579.46. CGST (6%) = ₹34.77. SGST (6%) = ₹34.77. Total = ₹649. Stored: orders.cgst_amount = 34.77, sgst_amount = 34.77, igst_amount = 0.	calculateGST()
6	Tax Invoice PDF for Mumbai customer	Shows: Taxable Value ₹579.46 | CGST (6%) ₹34.77 | SGST (6%) ₹34.77 | Total ₹649. HSN: 3406.	generateInvoicePDF()

Error / Edge Case	Handling
Cart has candles (12%) and room spray (18%)	Calculate GST per line item separately. Each item has its own hsn_code and gst_rate. Sum for invoice.

Samorah's registered state is wrong in settings	Affects every invoice. Super Admin must set settings.store_state correctly before first order. Admin → Settings → Tax/GST → Registered State.
Customer address has no state field	Validation: state is required field in address form. Block checkout if missing.

Claude Build Note:

Build a calculateGST(lineItems, customerState, storeState) function: for each item → extract taxable_value = price / (1 + gst_rate/100) → if customerState === storeState → cgst = igst = taxable_value * (gst_rate/2) / 100, else → igst = taxable_value * gst_rate / 100. Store all three columns. Tax Invoice PDF must show zero-value columns as '0.00' not blank.

SECTION C — Admin & Operations Flows (A1–A18)

A1 Add New Candle Product — 6-Step Form

Admin Flow — Product Goes Live in <60 Seconds

Actor	Manager or Admin	**Trigger**	Admin opens /admin/products/new to add 'Kashmiri Chai' candle
Preconditions	Product category 'Candles' exists. Collections 'Vol. I — Dessert Chapter' exists. Cloudinary connected.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Admin opens /admin/products/new	6-step wizard renders. Progress bar shows Step 1/6.	GET /admin/products/new
2	Step 1: Basic Info	Name: 'Kashmiri Chai'. Slug auto-generated: 'kashmiri-chai'. Category: Candles. Collection: Vol. I. Price: ₹649. Weight: 380g. HSN: 3406. GST: 12%. Short description. Status: Draft. Tags: bestseller, warm.	productStore.setBasicInfo()
3	Step 2: Variants	Admin clicks 'Add Variant'. Row: Name '100g Glass', SKU auto: SAM-CAN-001-100G-GL, Price ₹449, Stock 50, Vessel: Glass, Size: 100g, Barcode: (optional). Adds 2 more rows: 140g Ceramic, 180g Terracotta. 3 variants total.	productStore.addVariant()
4	Step 3: Images	Drag-drop 4 images. First → Primary. Admin adds alt text: 'Kashmiri Chai candle in ceramic vessel — warm spiced gourmand fragrance'. Images upload to Cloudinary, URLs stored.	Cloudinary upload + product_images INSERT
5	Step 4: Fragrance Notes	Top notes: Bergamot, Black Tea, Star Anise. Heart notes: Rose, Cardamom, Cinnamon. Base notes: Sweet Milk, Vanilla, Sandalwood.	fragrance_notes INSERT
6	Step 5: SEO	SEO Title: 'Kashmiri Chai Scented Candle — Warm Spiced Gourmand	productStore.setSEO()

		| Samorah' (63 chars). SEO Desc: 'Luxury handmade scented candle...' (158 chars). SERP preview shown below inputs.	
7	Step 6: Preview	Product card preview (as it appears on collection page). Basic product hero preview. Admin satisfied.	Client render
8	Admin clicks 'Publish'	products.status = active. POST /api/admin/products.	POST /api/admin/products
9	Next.js ISR revalidation triggered	revalidatePath('/shop/kashmiri-chai'), revalidatePath('/collections/dessert-chapter'). Cache purged.	next/cache revalidatePath()
10	Product live in <60 seconds	Customer visits /shop/kashmiri-chai — sees new product fully rendered.	GET /shop/kashmiri-chai

Error / Edge Case	Handling
Admin tries to publish without alt text on primary image	Step 3 validation: 'Primary image requires alt text'. Cannot advance to Step 4.
SKU auto-generated conflicts with existing	DB unique constraint fires. System auto-increments: SAM-CAN-001 → SAM-CAN-002. Admin sees new SKU.
Image upload fails (Cloudinary)	Error shown in Step 3. Upload retry button. Product cannot be published without at least 1 image.
Admin leaves halfway through (browser closes)	Product saved as Draft (status: draft). Admin can resume from Step 1 — all filled data persisted.

Claude Build Note:
Build the multi-step form with persistent draft state — save to DB as status=draft after each step using debounced auto-save. Never lose admin data on navigation. The SKU auto-generation function should query MAX(numeric portion of SKU per category) and increment by 1.

A2 Bulk CSV Product Upload — 10 to 30 Products

Admin Flow — Initial Catalog Population

Actor	Manager or Admin	**Trigger**	Admin uploads a CSV to add 15 initial Samorah candles at once
Preconditions	CSV template downloaded. Products sheet filled with all required columns. At least 1 image URL per product in Cloudinary.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Admin clicks 'CSV Import' on Products page	Modal opens. Step 1: Download Template button.	GET /admin/products/csv-template
2	Admin downloads CSV template	Template has columns: name, slug, category, collection, price, sale_price, weight, hsn_code, gst_rate, short_description, variant_name, variant_sku, variant_price, variant_stock, vessel_type, image_url_1 (primary), image_alt_1, seo_title, seo_description, tags	CSV download
3	Admin fills 15 rows in Excel/Google Sheets	Each row = one variant. Multiple rows per product (same name, different variant). Saves and uploads CSV.	Local
4	CSV uploaded	POST /api/admin/products/csv-import. File parsed with Papa Parse.	POST /api/admin/products/csv-import
5	Validation report shown	Row-by-row errors highlighted red. Example: Row 7: 'Duplicate SKU SAM-CAN-001-100G-GL'. Row 12: 'Missing HSN code'. Row 15: 'image_url_1 is not a valid Cloudinary URL'. 3 errors, 12 valid rows.	Validation engine
6	Admin fixes errors in CSV, re-uploads	Validation runs again. 0 errors. 15 valid rows (3 products × 5 variants).	POST again
7	Preview shown before confirming	Table: Product Name | Variant | SKU | Price | Stock | Action. All	Preview render

		15 rows with green 'Ready' badge.	
8	Admin clicks 'Confirm Import'	Atomic DB transaction: INSERT products, INSERT variants, INSERT product_images, INSERT fragrance_notes for all 15 rows. Rollback all on any failure.	DB transaction
9	ISR revalidation triggered for all new products	All new product pages and collection pages revalidated.	revalidatePath() × N
10	Admin sees success: '15 products imported successfully'	Products now appear in Products list and on store.	—

Error / Edge Case	Handling
Partial import fails (row 8 DB error)	Entire transaction rolled back. None of the 15 imports persist. Admin must fix and retry.
Image URLs not uploaded to Cloudinary yet	Validation fails for those rows. Admin must upload images first, get Cloudinary URLs, then add to CSV.
More than 100 rows uploaded	Admin warning: 'Large imports may take up to 2 minutes. Do not close this tab.'

Claude Build Note:
The CSV import must use an atomic PostgreSQL transaction — all-or-nothing. Use Papa Parse on frontend for pre-validation before upload. Build the validation engine as a separate service function (validateProductCSVRow) that tests each row against all business rules before any DB writes.

A7 NDR — Instruct Re-attempt Delivery

Admin Flow — Non-Delivery Report Management

WHY NDR MANAGEMENT IS CRITICAL FOR SAMORAH:

In India, 15–30% of COD orders fail delivery on first attempt. Without NDR tracking, these orders:

- Sit unnoticed for days → customer gets angrier → bad reviews
- Pile up as RTOs → product returned damaged → revenue lost
- Hurt Shiprocket service rating → higher rates charged

Actor	Manager or Admin	**Trigger**	Shiprocket courier fails to deliver — customer was unavailable or address was unclear
Preconditions	Shiprocket sends NDR webhook. orders.ndr_status = delivery_failed. Admin NDR tab shows the order.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Courier attempts delivery — customer unavailable	Shiprocket records: NDR Reason = 'Customer not available'. Timestamp logged.	Shiprocket courier app
2	Shiprocket sends NDR webhook to Samorah	POST /api/shiprocket/webhook. payload: { order_id, awb, ndr_status: 'delivery_failed', reason: 'customer_not_available' }	POST /api/shiprocket/webhook
3	Webhook updates DB	UPDATE orders SET ndr_status = 'delivery_failed', ndr_reason = 'customer_not_available'.	UPDATE orders
4	Admin dashboard: NDR alert badge	Dashboard shows: '2 orders need NDR attention'. Quick action: 'View NDRs'.	Admin dashboard widget
5	Admin opens Orders → NDR Tab	Table: Order# | Customer | Phone | AWB | Courier | Failure Reason | Date | Action buttons (Instruct Re-attempt | Accept RTO)	GET /admin/orders?tab=ndrs
6	Admin calls customer	'Hi, we tried delivering your Samorah order. Can we attempt again tomorrow?' Customer: 'Yes, I'll be home.'	External call

7	Admin clicks 'Instruct Re-attempt'	Modal: delivery instructions (e.g. 'Ring bell 3 times, call 5 min before'). Admin clicks Confirm.	—
8	API calls Shiprocket NDR endpoint	POST Shiprocket /orders/{shiprocket_order_id}/ndr/update: { action: 're_attempt', delivery_instructions: '...' }	POST Shiprocket NDR API
9	orders.ndr_status updated	= 're_attempt_scheduled'. NDR tab shows updated status.	UPDATE orders.ndr_status
10	Courier re-attempts next day	Delivery successful. Shiprocket webhook: status = delivered. ndr_status cleared.	Shiprocket webhook

Error / Edge Case	Handling
Customer says address is wrong (provide new address)	Admin updates address in Shiprocket dashboard directly. Then instructs re-attempt with new address note.
3rd re-attempt also fails	Accept RTO (flow A8). Package returns to Samorah warehouse.
COD amount not collected on re-attempt	Shiprocket tracks COD collection separately. Admin must reconcile in Shiprocket dashboard.

Claude Build Note:
Build the NDR tab as a separate filter view in /admin/orders: ?ndr=true. The 'Instruct Re-attempt' button must open a modal with a delivery notes field before calling Shiprocket API. Log all NDR actions in audit_logs: { action: 'ndr_reattempt_instructed', entity: 'order', entity_id, note }.

A10 Bulk CSV Inventory Update

Admin Flow — Restocking Multiple SKUs at Once

Actor	Manager or Admin	**Trigger**	Admin receives new stock batch — needs to update 12 SKUs simultaneously
Preconditions	New stock counts known for all variants. CSV template downloaded.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Admin opens /admin/inventory	Inventory dashboard. 'Bulk Update (CSV)' button in top right.	—
2	Downloads inventory CSV template	Template columns: variant_sku, new_stock_quantity, note (optional)	CSV download
3	Admin fills 12 rows in spreadsheet	Example: SAM-CAN-001-100G-GL, 75, 'New batch Oct 2025'	Local
4	Uploads CSV	POST /api/admin/inventory/bulk-update. Papa Parse reads file.	POST /api/admin/inventory/bulk-update
5	Validation report	Row-by-row check: SKU exists? New stock ≥ 0? Row 5: 'SKU SAM-CAN-999 not found'. 1 error, 11 valid.	Validation engine
6	Admin fixes row 5, re-uploads	0 errors. 12 valid rows. Preview table shown: SKU | Product Name | Current Stock | New Stock | Change.	—
7	Admin confirms	Atomic transaction: UPDATE variants SET stock = new_stock_quantity. INSERT inventory_logs for each row: change_type='restock', qty_change=(new-old), qty_after=new, note, admin_id.	DB transaction
8	Low stock alerts cleared	For any SKU where new stock > low_stock_threshold → alert status cleared in dashboard.	Dashboard refresh
9	Success message	'12 SKUs updated successfully. Total units added: 487.'	—

Error / Edge Case	Handling
Admin uploads 'add X units' instead of 'set to X units'	Two modes available: 'Set stock to' (absolute) and 'Add to stock' (relative). Admin must choose mode before upload. Default: Set stock to (absolute).
SKU not found in DB	Row rejected. All other rows proceed (non-atomic per row, not per batch). Admin can fix specific rows.

Claude Build Note:

Provide two modes in the bulk upload: 'Set Stock To [value]' (replaces current stock) and 'Add To Stock [value]' (increments current stock). The inventory_log must record both old and new values regardless of mode. This is the audit trail for GST/accounting purposes.

SECTION D — Marketing & Automation Flows (M1–M12)

M1 New Collection Launch Campaign

Marketing Flow — Vol. IV Nature Chapter Launch

Actor	Editor + Admin + Marketing Team	**Trigger**	Samorah is ready to launch Vol. IV — Nature Chapter with 4 new candles
Preconditions	Products created in Admin (status: draft). Blog post drafted. Banner images designed. Email list segmented.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Week -2: Teaser phase	Admin → Banners → Edit 'hero' slot: 'Something from Nature is coming' + moody forest image. Update announcement bar: 'Vol. IV — Nature Chapter launches in 14 days'. No products visible yet.	homepage_banners UPDATE
2	Week -1: Pre-launch email	Newsletter campaign: 'Introducing Vol. IV — The Nature Chapter. A fragrance world shaped by earth, rain, and forest.' No product links yet. CTA: 'Set a reminder'. open rate tracked.	Resend Broadcasts
3	Launch day: Products published	Admin → Products → Select all 4 draft products → Bulk action: 'Publish'. status = active. ISR revalidation.	UPDATE products.status = active + revalidatePath()
4	Launch day: Collection page updated	Admin → Collections → Vol. IV → Assign 4 new products. Collection page now shows new candles.	UPDATE collections
5	Launch day: Blog post published	Admin → Blog → Vol. IV story → Click 'Publish Now'. Sanity CMS or MDX post goes live.	Blog publish
6	Launch day: Homepage hero updated	Admin → Banners → hero: 'Vol. IV — Nature Chapter is here' + nature candle editorial image +	homepage_banners UPDATE + ISR

		'Explore Now' CTA → /collections/nature-chapter.	
7	Launch email sent	Newsletter: 'Vol. IV is live. 4 new fragrances: Forest Rain, Vetiver Clay, Wild Fig, Stone & Moss.' Product cards with CTA. Sent to full subscriber list.	Resend Broadcasts
8	GA4 monitoring	Events: view_item_list (collection page surge), add_to_cart (new products), purchase. Dashboard shows real-time product performance.	GA4 Dashboard
9	Week +1: Low stock check	If any Vol. IV variant hits low_stock_threshold → admin alert email → restock before stockout.	pg_cron + Resend alert

Error / Edge Case	Handling
Blog post auto-scheduled but fails to publish	Sanity CMS or pg_cron scheduling failure. Admin must manually publish. Build webhook from Sanity to trigger ISR on publish.
Vol. IV products accidentally made active before launch day	Admin must use 'Scheduled Publish' feature or keep status=draft until launch moment.

Claude Build Note:
The 'Scheduled Publish' feature for products is a Phase 2 addition — implement publish_at TIMESTAMP on products table. pg_cron sweeps: UPDATE products SET status='active' WHERE publish_at <= NOW() AND status='draft'. Until then, admin manually publishes on launch day.

M5

Abandoned Cart — pg_cron Trigger Deep Dive

Marketing Flow — Technical Implementation

Actor	pg_cron System (Supabase) + Resend	**Trigger**	Hourly cron job identifies abandoned carts and triggers email sequences
Preconditions	pg_cron enabled in Supabase. Resend API configured. cart table has: last_activity_at, abandoned_email_1_sent, abandoned_email_2_sent, abandoned_email_3_sent, email (for guests).		

SQL Function in Supabase (runs every hour via pg_cron):

```
CREATE OR REPLACE FUNCTION sweep_abandoned_carts() RETURNS void AS $$
BEGIN
  -- Email 1: 1 hour after last activity, not sent yet
  FOR cart_row IN SELECT * FROM cart WHERE last_activity_at < NOW()-'1
hour'::interval
    AND abandoned_email_1_sent = false
    AND NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = cart.user_id
      AND o.created_at > cart.last_activity_at) LOOP
    PERFORM http_post(edge_function_url, cart_row);  -- triggers Resend
    UPDATE cart SET abandoned_email_1_sent=true WHERE id=cart_row.id;
  END LOOP;
  -- Repeat for Email 2 (24hr) and Email 3 (72hr + coupon auto-generate)
END; $$ LANGUAGE plpgsql;
```

Step	Action (Who Does What)	System Response	DB / API Call
1	pg_cron runs every hour	SELECT cron.schedule('0 * * * *', 'SELECT sweep_abandoned_carts()');	Supabase pg_cron
2	Email 1 cohort identified	Carts with last_activity > 1hr ago, email_1 not sent, no recent order. Guest carts need email field populated.	pg_cron SQL query
3	Supabase Edge Function triggered	For each cart: POST to Edge Function with cart items data. Edge Function calls Resend API.	Supabase Edge Function
4	Email 1 sent: 'Your candles are waiting'	Soft emotional. No discount. Product images. Return to cart CTA.	POST Resend API
5	Flag set: abandoned_email_1_sent = true	Prevents Email 1 from being sent again on next cron run.	UPDATE cart

6	24hr later: pg_cron identifies Email 2 cohort	Same logic but checks abandoned_email_2_sent = false and last_activity > 24hr.	pg_cron SQL query
7	Email 2 sent: 'Still thinking?'	Brand story excerpt. Product shown. No discount still.	POST Resend API
8	72hr later: Email 3 with auto-coupon	Auto-generate coupon: code=CART10-[uuid], value=10%, max_uses=1, expires=NOW()+48hr. Link coupon to cart session/user.	INSERT coupons + POST Resend
9	Customer clicks Email 3 link	Cart restored. Coupon CART10-xxx pre-applied in cartStore. 'Special offer applied' shown.	GET /cart?coupon=CART10-xxx

Claude Build Note:
The Supabase Edge Function approach avoids exposing the Resend API key in the pg_cron SQL function. pg_cron calls the Edge Function URL with a secret header (shared secret auth). Edge Function has Resend key in env vars. This is the correct security pattern.

M12 Search Analytics — Zero Result Queries

Marketing Flow — Discovering Missing Products

Actor	Customer + Admin	**Trigger**	Customer searches for something Samorah doesn't sell yet — zero results returned
Preconditions	search_logs table exists with columns: query, results_count, clicked_product_id, user_id, session_id, created_at		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer opens search bar	useSearchStore.openSearch(). Search overlay appears.	useUIStore
2	Customer types 'rose candle'	Debounced query — after 300ms pause: GET /api/search?q=rose+candle	GET /api/search
3	Search executes full-text query	PostgreSQL: SELECT * FROM products WHERE to_tsvector(name || ' ' || description) @@ to_tsquery('rose & candle') AND status='active'. Result: 0 rows.	PostgreSQL FTS
4	Zero results returned	UI shows: 'No results for "rose candle"'. Suggestions shown: 'Try: Kashmiri Chai, Floral Chapter'.	—
5	Zero result logged	INSERT search_logs: { query: 'rose candle', results_count: 0, clicked_product_id: null, user_id, session_id, created_at: NOW() }	INSERT search_logs
6	Customer clicks 'Kashmiri Chai' suggestion	UI navigates to product. INSERT search_logs: { query: 'rose candle', results_count: 0, clicked_product_id: 'kashmiri-chai-uuid' }	INSERT search_logs + UPDATE
7	Admin reviews search analytics monthly	Admin → Analytics → Search Insights. Table: Top zero-result queries | Top clicked products | Search conversion rate.	GET /admin/analytics/search

8	Admin spots pattern	'rose candle' searched 47 times in Nov. 'oud candle' searched 31 times. Both = 0 results. Decision: create these products.	Business decision

Claude Build Note:
The search_logs table is a product roadmap goldmine. Build the admin analytics view with: (1) Top 20 zero-result queries this month, (2) Top 20 queries by volume, (3) Top 20 queries with click-throughs. Export to CSV. Run this analysis before every new collection launch to understand what customers want.

SECTION E — Security & Edge Case Flows (S1–S8)

S1 Coupon + Loyalty Stacking — API-Level Block

Security Flow — Discount Stacking Rule Enforcement

STACKING RULES (enforce at API, not just UI):

❌ Coupon Code + Loyalty Points redemption = BLOCKED

❌ Two coupon codes = BLOCKED (second replaces first, or show error)

✅ Gift Card + Coupon Code = ALLOWED (gift card is pre-paid currency)

✅ Gift Card + Loyalty Points = ALLOWED

✅ Gift Card + Gift Card = BLOCKED in *"Phase 1 Limitation: Only one gift card code can be applied per order.*

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer has 500 loyalty points queued on checkout	checkoutStore.loyaltyPointsUsed = 500. ₹50 discount applied.	—
2	Customer enters SUMMER15 coupon code	POST /api/coupons/apply. Coupon is valid (15% off, active, not expired).	POST /api/coupons/apply
3	API checks stacking rule	if (request.loyaltyPointsUsed > 0 && couponCode !== null) → CONFLICT	Server stacking check
4	API returns 409 Conflict	{ error: 'discount_stack_conflict', message: 'Cannot combine coupon with loyalty points. Choose one.' }	HTTP 409
5	UI shows selection modal	Two clear options: 'Keep ₹50 loyalty discount' vs 'Apply SUMMER15 (save ₹97)'. User picks SUMMER15.	Modal component
6	Loyalty redemption cancelled	checkoutStore.loyaltyPointsUsed = 0. Coupon applied. Discount = ₹97.	checkoutStore update
7	Checkout finalize — server double-check	POST /api/checkout/finalize validates: not both coupon + loyalty points. If both present (UI	Server validation

		bypass attempt) → reject order with 400.	

Error / Edge Case	Handling
Clever user inspects network calls and sends both in API request	The /api/checkout/finalize endpoint enforces the rule server-side. Cannot be bypassed via API manipulation.
Admin creates a 'loyalty stackable' coupon — is that possible?	Not in Phase 1. Coupon entity has no stackable_with_loyalty boolean. All coupons follow the rule. Phase 2 consideration.

Claude Build Note:
Always enforce discount rules in TWO places: (1) /api/coupons/apply for immediate feedback, (2) /api/checkout/finalize as the authoritative gate. Never trust the frontend to enforce business rules alone. Log any attempts to bypass stacking rules in audit_logs: { action: 'discount_stack_attempt' }.

S7 Webhook Logs — Debugging Razorpay Errors

Security Flow — Observability & Debugging

webhook_logs table schema:

```
id UUID | provider VARCHAR(50) | event_type VARCHAR(100) | payload JSONB |
status ENUM(received|processed|failed|duplicate) | error_message TEXT |
retry_count INTEGER | processed_at TIMESTAMP | created_at TIMESTAMP
```

Actor	System (Razorpay webhook) + Admin	**Trigger**	A Razorpay payment.captured webhook fails to fully process — Shiprocket API is down
Preconditions	Payment has been captured. Webhook received. But Shiprocket API returns 503.		

Step	Action (Who Does What)	System Response	DB / API Call
1	Webhook received	POST /api/razorpay/webhook. HMAC verified. INSERT webhook_logs: { provider:'razorpay', event_type:'payment.captured', payload:{...}, status:'received' }	INSERT webhook_logs
2	Idempotency check passed	No duplicate. Proceed to process.	SELECT orders
3	Order status updated, inventory decremented	Orders updated. Inventory decremented. Success so far.	UPDATE orders + variants
4	Shiprocket API call fails (503 Service Unavailable)	POST Shiprocket /orders/create returns 503.	Shiprocket API → 503
5	Error logged	UPDATE webhook_logs SET status='failed', error_message='Shiprocket 503: Service Unavailable', retry_count=0.	UPDATE webhook_logs
6	Email still sent (email is independent)	Order confirmation email sent despite Shiprocket failure. Customer gets confirmation.	POST Resend API
7	Retry logic (3 attempts, exponential backoff)	Supabase Edge Function retries Shiprocket: 5min later →	Edge Function retry

		fail → 15min → fail → 60min → success.	
8	On success	UPDATE webhook_logs SET status='processed', retry_count=3. UPDATE orders SET shiprocket_order_id='...'. Admin dashboard: no NDR alert.	UPDATE webhook_logs + orders
9	If all retries fail	webhook_logs.status = 'failed' permanently. Admin alert email: 'Shiprocket sync failed for order SAM-2025-0042. Manual action required.' Admin creates Shiprocket order manually.	Resend admin alert

Error / Edge Case	Handling
How does admin find failed webhooks?	Admin → /admin/webhooks: table of all webhook_logs filtered by status='failed'. One-click 'Retry' button per row.
Resend email also fails (email service down)	Separate Resend webhook log entry. Customer order is safe in DB. Email can be resent manually from admin order detail.
Multiple webhook providers (Razorpay + Shiprocket)	Same webhook_logs table, differentiated by provider field. Both visible in admin webhook log view.

Claude Build Note:
Build /admin/webhooks as a read-only diagnostic tool accessible to Admin and Super Admin. Columns: Timestamp | Provider | Event | Status | Retry Count | Error | Action (Retry button). This screen alone will save hours of debugging when integrations fail. Every webhook endpoint should write to webhook_logs as its FIRST action.

S8 Product Recommendations — Fragrance Family & Related

Security/Feature Flow — Recommendation Engine

Actor	Customer viewing a product page	**Trigger**	Customer is viewing Kashmiri Chai (fragrance_family: Dessert/Gourmand, mood: Warm, collection: Vol. I)
Preconditions	related_products table OR query-based recommendation logic implemented		

Step	Action (Who Does What)	System Response	DB / API Call
1	Customer views Kashmiri Chai product page	Page loads with 'You May Also Like' section at bottom.	GET /shop/kashmiri-chai
2	Recommendation query executes	Rule 1 (same collection, different product): SELECT * FROM products WHERE collection_id = 'vol-1' AND id != current_product_id LIMIT 3.	getRelatedProducts(product)
3	Rule 2 (same fragrance family)	SELECT * FROM products WHERE fragrance_family = 'dessert_gourmand' AND id != current_product_id LIMIT 3.	—
4	Rule 3 (same mood)	SELECT * FROM products WHERE mood = 'warm' AND id != current_product_id LIMIT 2.	—
5	Deduplicate and rank	Combine results: products appearing in 2+ rules ranked higher. Final: max 6 recommendations shown.	Client-side dedup
6	Rendered as product cards	Gallery-style product cards. Clicking fires select_item GA4 event with item_list_name: 'recommendations'.	GA4 event
7	Search_logs enhancement	If customer searched before landing here, log: { query: 'warm chai candle', clicked_product: 'kashmiri-chai-id' } — helps improve recommendations.	INSERT search_logs

New Database Tables Needed	Purpose
related_products	Manual overrides: product_id, related_product_id, sort_order — set by admin for editorial curation
search_logs	query, results_count, clicked_product_id, user_id, session_id, created_at
wishlists	user_id, product_id, variant_id, created_at — persistent DB-backed wishlist
stock_reservations	variant_id, quantity, session_id, expires_at — prevents overselling
webhook_logs	provider, event_type, payload, status, error_message, retry_count
notifications	user_id, title, body, type, is_read, created_at — persistent notification center

Claude Build Note:
Add these 6 tables to the Supabase migration in Phase 4 (Database). They are referenced throughout multiple flows. The recommendations engine starts as a simple rule-based SQL query (Phase 1). Phase 2: add ML-based collaborative filtering using order history data.

Appendix A — 6 Missing Database Tables (Add to BRD Phase 4)

These tables were identified during USD creation and must be added to the Supabase migration.

Reference: Add all 6 to BRD Section 8 Phase 4 migration file.

Table	Key Columns	Why It Exists
stock_reservations	variant_id, quantity, session_id, expires_at	15-minute checkout hold. Prevents overselling when 2 customers check out same last item simultaneously. Cleared by pg_cron every 5 min.
webhook_logs	provider, event_type, payload JSONB, status, error_message, retry_count	Razorpay + Shiprocket webhook observability. Saves hours of debugging. Admin /admin/webhooks screen reads this table.
search_logs	query, results_count, clicked_product_id, user_id, session_id, created_at	Product discovery analytics. Zero-result queries = product roadmap. Monthly analysis drives new collection decisions.
related_products	product_id, related_product_id, sort_order, relation_type (same_family/same_mood/manual)	Powers 'You May Also Like' section on product page. Admin can manually curate editorial pairings.
wishlists	user_id, product_id, variant_id, created_at	Zustand wishlist store already exists but needs DB persistence for logged-in users. Enables back-in-stock campaigns and wishlist email reminders.
notifications	user_id, title, body, type (order/loyalty/promo), is_read, action_url, created_at	Persistent notification center in /account. Shows order updates, tier upgrades, back-in-stock alerts. Zustand state is ephemeral — DB makes it persistent.

Appendix B — 3 BRD Updates from Final Review

#	Section to Update	Exact Change Required
1	BRD Section 11.5 + 15.1	Add stacking rule: 'Only one coupon per order. Coupons cannot be combined with loyalty point redemptions. Gift cards are exempt from this rule and may be combined with either.'
2	BRD Section 12.2	Add note: 'The /api/razorpay/verify endpoint exists solely to redirect the customer to the Thank You page. ALL business logic (inventory, Shiprocket, email, GST, invoice) runs exclusively in /api/razorpay/webhook. Never put order processing logic in /verify.'
3	BRD Section 22.2 (Future Deliverables)	Add: 'WhatsApp API Integration via Wati or Interakt — for OTP delivery and automated Shiprocket tracking updates. Indian customers expect order updates on WhatsApp. Phase 2 implementation after core platform is stable.'

Appendix C — Claude Prompts for New USD Flows

Flow	Claude Prompt Template
C10	'Build the coupon application API at /api/coupons/apply. Enforce stacking rule from SAMORAH USD C10: if request body contains both coupon_code and loyalty_points_used > 0, return HTTP 409 with error discount_stack_conflict. Also enforce in /api/checkout/finalize as server-side double-check.'
P5	'Build the Razorpay webhook handler at /api/razorpay/webhook. All business logic goes HERE — not in /verify. Follow SAMORAH USD P5 exactly: 1) HMAC verify 2) idempotency check 3) order update 4) inventory decrement 5) Shiprocket create 6) GST calculate 7) invoice PDF 8) email send.'
A7	'Build the NDR management tab in the admin orders section. Follow SAMORAH USD A7: filter by orders WHERE ndr_status = delivery_failed. Show: Order# | Customer Phone | AWB | Reason | Date. Two action buttons: Instruct Re-attempt (calls Shiprocket NDR API) and Accept RTO. Log all actions in audit_logs.'
S7	'Create the webhook_logs table from SAMORAH USD S7 schema. Build /admin/webhooks screen: TanStack Table showing all logs filtered by status=failed. One-click Retry button per row. Also build the Edge Function retry logic with 3 attempts at 5min, 15min, 60min intervals.'
P7	'Create the stock_reservations table from SAMORAH USD P7. Build the reserve-stock API at /api/checkout/reserve that creates a 15-minute hold. Build the pg_cron cleanup job that runs every 5 minutes to DELETE FROM stock_reservations WHERE expires_at < NOW(). Update all available-stock queries to subtract active reservations.'